PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART I

Software Verification Architecture

Version 1.2 Amendment Edition

Amendment purpose

Make verification efficiency a correctness property. Version 1.2 defines minimum sufficient evidence, semantic invalidation ceilings, immutable evidence preservation, runtime-conformance proof, gate-specific proof depth, and verification-throughput quality rules so Sounding Line remains deep and deterministic without ritual re-execution of proof that has not changed in meaning.

Effective August 11, 2026 upon protected-mainline acceptance | Document ID CS-SL-P1-001 v1.2

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part I, Version 1.2 Amendment Edition
Base authority	Version 1.0, July 28, 2026, plus the accepted Version 1.1 Amendment Edition dated August 10, 2026.
Version	1.2
Effective date	August 11, 2026 upon acceptance through the protected Sounding Line mainline path.
Document ID	CS-SL-P1-001 v1.2
Status	Governing amendment candidate; becomes authoritative on protected-mainline acceptance. Additive and superseding only where clauses are explicitly named.
Amendment trigger	Observed growth in authoritative-run wall time and repeated setup/revalidation during the August 2026 concurrent project wave, including suite-per-runner setup repetition, broad exclusive serialization, browser provisioning duplication, and late expensive failure discovery.
Primary objective	Maximize verification throughput without reducing coverage, evidence integrity, source binding, risk floors, or release confidence.

Normative incorporation rule

Version 1.0 and Version 1.1 remain in force except where this Version 1.2 amendment explicitly adds, clarifies, or supersedes a clause. Version 1.2 is not permission to run fewer required tests or accept weaker evidence. It requires Sounding Line to obtain the same or stronger decision confidence with the least redundant execution that can be proven correct. Where Version 1.1 permits a conservative broad rerun because automated preservation is not yet implemented, Version 1.2 permits that only as an explicit fail-closed fallback with recorded implementation debt; blanket invalidation may not remain the normal steady state.

The three-document authority split remains unchanged: Part I defines verification meaning, Part II defines execution machinery, and Part III defines repository, Codex, CI, mainline, and release enforcement.

1. Amendment Summary and Supersession Map

Version 1.1 established semantic invalidation, carry-forward evidence, closure efficiency, and failure-class repair boundaries. Version 1.2 closes the remaining gap between those semantics and the runtime by defining efficiency as part of verification correctness and by making proof minimization, resource justification, runtime conformance, and gate-scoped evidence first-class requirements.

Existing authority area	Version 1.2 addition
1.2 Verification Promise	Adds the Proof Efficiency Invariant: an optimized run must satisfy every mandatory obligation of a correct unoptimized run, with equal or stronger evidence semantics.

Existing authority area	Version 1.2 addition
4 Core Domain	Adds Minimum Sufficient Evidence Set, Verification Obligation, Runtime Conformance Receipt, Prepared Artifact Identity, Overprovisioning, Overvalidation, and Conservative Fallback.
5 Design Principles	Requires least-redundant correct proof, no setup work without declared resource need, and no global serialization without a proven shared resource.
14 Test Impact Analysis	Makes inverse impact analysis mandatory before broad rerun and introduces invalidation ceilings for test-only, record-only, and unrelated-mainline changes.
16 Status Language	Adds PRESERVED, REBOUND, INVALIDATED, FRESH, CONSERVATIVE_FALLBACK, and CONFORMANCE_FAILED decision states.
19 Evidence Architecture	Adds derived evidence-chain rules for prepared immutable artifacts, bundled workers, carry-forward, and runtime-conformance receipts.
20 Release Confidence	Defines gate-specific confidence composition across fresh and preserved evidence without weakening source/policy/environment binding.
25 Governance	Adds verification-efficiency quality metrics, runtime-conformance acceptance, and mandatory remediation of systematic overvalidation/overprovisioning.

Hard boundary

Version 1.2 is not a speed-over-correctness amendment. It forbids two opposite failure modes: under-validation, where required proof is missing or stale; and over-validation, where evidence is rerun or infrastructure is prepared without a semantic dependency that justifies the cost. Sounding Line must be able to explain both what it runs and what it does not rerun.

2. New Canonical Vocabulary

Term	Definition
Proof Efficiency Invariant	For a given gate and source state, any optimized execution must satisfy the same mandatory verification obligations and confidence semantics as a correct unoptimized execution.
Verification Obligation	A mandatory gate requirement that must resolve to one valid producer: fresh receipt, valid carry-forward, or approved rebinding record.
Minimum Sufficient Evidence Set	The smallest complete set of valid current evidence that satisfies every mandatory verification obligation for the requested gate.
Evidence Dependency Fingerprint	Canonical digest of the production contracts, tests, fixtures, schemas, toolchain, resources, environment class, and policy elements on which a receipt depends.
Prepared Artifact Identity	Source-independent or source-qualified identity of an immutable dependency environment, browser environment, generated client, or certified database baseline that can be reused safely.
Runtime Conformance Receipt	Evidence produced by Sounding Line proving that the plan and execution obey resource, scheduling, preservation, gate-scope, and efficiency contracts.

Term	Definition
Overprovisioning	Preparing or allocating expensive resources that are not declared or transitively required by the selected node/bundle.
Overvalidation	Executing evidence obligations whose dependencies have not changed and that could have been satisfied by valid preservation/rebinding.
Conservative Fallback	Fail-closed broad rerun used only when Sounding Line cannot prove safe preservation. It requires an explicit reason, scope, and implementation-debt record.
Bundle Worker	One runtime worker that executes multiple compatible suites while preserving independent per-suite receipt boundaries.
Gate Scope Ceiling	Maximum normal evidence class a narrow gate may select absent a documented semantic escalation.
Preservation Ratio	Mandatory obligations satisfied by valid carry-forward or rebinding divided by all obligations eligible for preservation.

3. Proof Efficiency Invariant

Sounding Line SHALL optimize for correctness first and redundancy second. “Fast” is valid only when the resulting decision is indistinguishable in required proof strength from a correct deeper execution. The planner, runtime, and finalizer therefore share one invariant: no optimization may delete a mandatory obligation, lower a contract risk floor, skip a required negative assertion, accept a stale receipt, or weaken cleanup/source/environment binding.

3.1 Allowed optimization dimensions

- Execution order, worker assignment, sharding, batching, resource lane, cache use, and concurrency may change when evidence meaning is preserved.
- Immutable prepared environments and certified baselines may be reused when their complete identity matches the current requirement.
- Prior evidence may be carried forward only through an auditable semantic invalidation analysis.
- Record-only closure may use a narrow gate when executable behavior and gate semantics are unchanged.
- Independent work may run concurrently when resource ownership proves isolation.

3.2 Forbidden optimization dimensions

- Dropping tests or negative cases solely because they are slow.
- Treating historical green status as current proof without source/policy/environment reconciliation.
- Replacing end-to-end coverage with unit coverage when the protected contract remains journey-level.
- Relaxing timeouts, retry counts, assertions, authorization checks, accessibility checks, or mutation/cleanup checks to improve throughput.
- Sharing mutable databases, ports, storage roots, build outputs, or browser state across runs without an explicit isolation contract.

> **Decision equivalence rule**
>
> A planner optimization is acceptable only when the finalizer could receive the same set of mandatory protected-contract claims, with current provenance, from either the optimized or an unoptimized correct execution. If equivalence cannot be proven, Sounding Line must fall back conservatively and record why.

4. Minimum Sufficient Evidence Set

The planner SHALL resolve each gate into explicit verification obligations before choosing execution. It then maps each obligation to one of three producer classes: fresh execution, preserved evidence, or rebinding. The output is the Minimum Sufficient Evidence Set (MSES).

```
GATE REQUEST
  -> mandatory obligations
  -> dependency fingerprint for each obligation
  -> current evidence search
      FRESH valid?       -> consume fresh receipt
      PRESERVED valid?   -> create/consume carry-forward receipt
      REBOUND valid?     -> regenerate source-bound record
      otherwise          -> schedule fresh execution
  -> verify every obligation has exactly one current producer
  -> finalizer
```

4.1 Planner completeness requirements

1. Enumerate every mandatory contract/suite/decision obligation required by the gate and policy version.
2. Record why each obligation is required and which source, policy, fixture, resource, or environment dependency makes it relevant.
3. Record why each omitted suite or evidence family is not required. Omission reasoning is part of the plan receipt.
4. Detect duplicate producers and resolve them deterministically; stronger current evidence may supersede weaker evidence but may not erase history.
5. Reject a plan in which a mandatory obligation has no current producer path.

4.2 Proof minimization is mandatory, not advisory

If two plans provide equal required confidence, Sounding Line SHOULD choose the plan with lower redundant setup and execution cost, provided resource and queue policy do not create a larger critical path. A broader plan is legal only when its broader dependency explanation is recorded and machine-verifiable.

> **No “run everything just in case” steady state**
>
> A broad full-gate fallback may be used to fail closed while missing automation is repaired, but the fallback must be tagged CONSERVATIVE_FALLBACK, attributed to a specific unsupported preservation/invalidation capability, and opened as Sounding Line implementation debt. It may not become the permanent answer to ordinary test-only, record-only, or unrelated-mainline changes.

5. Semantic Evidence Invalidation v1.2

Version 1.1 established that SHA inequality is not semantic invalidation. Version 1.2 requires the invalidation engine to classify each prior mandatory obligation deterministically and to expose the dependency path used for the classification.

Disposition	Meaning	Decision behavior
FRESH	A current-source receipt already exists and exactly matches required policy/environment identity.	Use directly.
PRESERVED	Prior semantic proof is unchanged; new-source carry-forward receipt proves unchanged dependency fingerprint.	Use via derived carry-forward receipt.
REBOUND	Semantic proof is unchanged, but source-bound	Regenerate/rebind and validate.

Disposition	Meaning	Decision behavior
	generated record/metadata must be regenerated.	
INVALIDATED	One or more protected dependencies changed.	Schedule fresh proof.
SUPERSEDED	A newer/stronger current receipt replaces it.	Retain historically; do not count twice.
CONSERVATIVE_FALLBACK	System cannot prove safe preservation even though broad invalidation is not semantically established.	Rerun safely, record implementation debt and reason.

5.1 Mandatory invalidation inputs

- Production files and direct/transitive consumers of protected contracts.
- Test files, shared helpers, assertion libraries, fixtures, test registrations, and owner metadata.
- Schemas, migrations, generated clients, database baseline identity, seed/fixture builders, and persistence semantics.
- Package lock, Node/runtime version, Playwright/browser versions, compiler/build configuration, and adapter identity when the receipt depends on them.
- Authorization/privacy/security/progression contracts and any projection that changes what a user or operator may observe or mutate.
- Sounding Line policy, gate definitions, risk floors, resource contracts, and finalizer semantics.
- Environment identity only for obligations whose claims depend on that environment class.

5.2 Default invalidation ceilings

Change	Default ceiling	Broaden only when
Single E2E locator/assertion repair	Owning browser suite + directly dependent decision evidence.	Broaden only if shared helper, fixture, route contract, or cross-suite assertion library changed.
Unit/component test repair	Owning suite + consumers of changed fixture/helper.	Broaden only through explicit shared dependency.
Fixture/seed builder change	All suites consuming that fixture/baseline + dependent decision nodes.	Schema/persistence consumers if authoritative seeded state changed.
Feature Catalog / completion status only	Catalog/docs/policy/record-integrity proof.	Executable matrix only if record changes executable gate semantics.
Unrelated project merged to main	Shared contract/schema/harness changes + exact integration proof.	All-project rerun only if dependency graph proves broad shared impact.
Sounding Line runtime code changed	Sounding Line conformance/self-tests + impacted execution families.	Product suites only when runtime change can alter their execution/evidence semantics.

6. Carry-Forward, Rebinding, and Derived Evidence

Historical receipts remain immutable. Preservation creates a new current-source derivation record. The finalizer must be able to traverse from the current decision through every derived receipt to the original proof and the exact invalidation analysis that preserved it.

Derived field	Requirement
newSourceSha	Exact current candidate or integrated SHA receiving the evidence.
priorReceiptDigest	Digest of immutable prior receipt.
priorSourceSha	Source state on which the original proof executed.

Derived field	Requirement
protectedObligations	Contract/suite/decision obligations supplied by the prior receipt.
dependencyFingerprintBefore / After	Canonical dependency identities used to prove equivalence.
changedIntervalDigest	Digest of the source/policy/environment delta examined.
disposition	PRESERVED, REBOUND, INVALIDATED, SUPERSEDED, or CONSERVATIVE_FALLBACK.
reasonCode / explanation	Machine-readable reason plus auditable human explanation.
plannerPolicyDigest	Exact policy/planner authority producing the disposition.
createdAt / actor	Auditable creation identity.

6.1 Preservation expiry

- A changed protected contract, fixture, schema, runtime adapter, toolchain dependency, or environment requirement invalidates preservation when the receipt depends on it.
- A policy version may declare a prior evidence family incompatible with new semantics; the prior receipt remains historical but cannot satisfy the new obligation.
- If dependency identity cannot be reconstructed or verified, fail closed and rerun the affected obligation. Do not fabricate equivalence.

7. Resource and Setup Evidence Semantics

Version 1.2 treats unnecessary setup as a verification-architecture defect because setup can dominate wall time, increase queue exposure, and create more opportunities for environmental failure without adding confidence. A node or bundle SHALL prepare only resources it declares or transitively requires.

Declared resource	Permitted preparation	Prohibited implicit preparation
node-slot / vitest-worker-pool	Node dependency environment and test runtime.	Database, browser engines, app server, or storage roots unless separately declared.
prisma-sqlite-client	Prisma client identity/schema validation as required.	Full seeded database unless sqlite-clone is also declared.
sqlite-clone	Certified baseline or migration-built run-owned clone.	Shared mutable database across independent runs.
browser-chromium	Exact Chromium engine/cache identity and run-owned browser context.	WebKit unless declared.
browser-webkit	Exact WebKit engine/cache identity and run-owned context.	Chromium unless declared.
production-build-directory	Source-bound build output identity.	Browser/database resources unless required by the protected journey.

Runtime overprovisioning rule

If a pure unit/static node triggers database migration/seed, browser downloads, application-server startup, or another undeclared expensive resource solely because a generic worker template does so, Sounding Line is nonconformant. The proof may still be functionally valid, but the runtime must emit RUNTIME_OVERPROVISIONING and the infrastructure owner must repair the worker path.

7.1 Immutable reuse is permitted when identity is complete

Dependency environments, browser binaries, generated clients, and certified database baselines may be reused across runs only when their complete identity is included in evidence and their mutability boundary is enforced. Mutable run state must still be cloned or allocated per run.

8. Gate-Specific Evidence Depth

Gate	Purpose	Normal proof depth	Hard limit
local-change	Fast impact-selected developer/Codex feedback.	Changed contracts + direct/transitive risks.	Cannot claim mainline or release readiness.
subsystem	Complete proof for a project/subsystem boundary before mainline candidacy.	Subsystem unit/component/browser/migration/privacy/etc. as required.	Cannot replace repository integration proof.
mainline	Protected integration authority for executable changes.	Minimum sufficient current evidence for all mandatory mainline obligations, fresh + preserved + rebound, plus runtime conformance and cleanup.	No missing mandatory evidence; no record-only shortcut for executable change.
record-closure	Post-merge record/status/provenance closure when executable behavior and gate semantics are unchanged.	Docs/catalog/receipt/index/policy integrity actually affected.	Browser/build/database matrix unless semantic dependency proves it.
release-candidate	Deep pre-release assurance across current product surface.	All release-candidate mandatory obligations, including provider/owner/manual gates where required.	Cannot be weakened because mainline was green.

> **Gate meaning rule**
>
> Gate classes change the required evidence set, not the meaning of a protected contract. A narrow gate may not be used to avoid proof that belongs to a broader claim. A broader gate may not be rerun merely because it is familiar.

9. Failure Classification and Legal Rerun Scope

Class	Legal repair scope	Default rerun scope
PRODUCT_DEFECT	Smallest responsible production scope; regression evidence; invalidate affected obligations.	Repair product and rerun affected proof.
TEST_DEFECT	Owning test/assertion/helper/fixture only.	Rerun owning suite + directly affected evidence.
FIXTURE_DEFECT	Fixture/seed/baseline owner.	Rerun all consumers of changed fixture/baseline.
ENVIRONMENT_DEFECT	Environment/reference-runner owner.	Reproduce/rerun environment-sensitive node; preserve unrelated semantic proof.
INFRASTRUCTURE_DEFECT	Sounding Line runtime/broker/executor owner.	Repair runtime; rerun only evidence whose execution semantics may have been affected.
INFRASTRUCTURE_CONTENTION	Scheduler/broker/resource queue.	Wait/reroute; product/test edits forbidden.
TIMEOUT	Diagnose product no-progress vs test wait vs environment/setup/controller budget.	Only the diagnosed owner may change its budget/behavior.
STALE_TEST	Test/contract owner aligns assertion with accepted product authority.	Preserve product; rerun affected test family.

10. Runtime Conformance as Verification Evidence

Sounding Line SHALL verify its own execution behavior. Runtime conformance is not a second authority and does not create a competing release gate. It is a mandatory evidence obligation inside Sounding Line whose receipt is consumed by the existing finalizer.

Conformance code	Meaning
RUNTIME_OVERPROVISIONING	Worker prepared an expensive resource not declared/transitively required.
UNJUSTIFIED_GLOBAL_SERIALIZATION	Independent nodes were serialized by a broad category rather than an actual shared resource contract.
BROWSER_ENGINE_SCOPE_VIOLATION	Worker installed or required browser engines not declared by selected node/bundle.
FIXED_PORT_ARCHITECTURE_VIOLATION	Run relies on a universal fixed port as a global mutex instead of a leased run-owned endpoint.
EVIDENCE_INVALIDATION_MANIFEST_MISSING	Broad or preserved decision attempted without required semantic invalidation analysis.
OVERINVALIDATION_UNEXPLAINED	Planner reruns broad evidence without a dependency path explaining the expansion.
RECORD_GATE_SCOPE_VIOLATION	Record-only change selects executable browser/build/database proof without semantic justification.
BUNDLE_RECEIPT_BOUNDARY_INVALID	Bundled worker cannot prove independent per-suite receipt completeness/counts.
PREPARED_ARTIFACT_IDENTITY_MISMATCH	Cached/baseline artifact identity does not match declared requirement.

10.1 Conformance severity

- Evidence-integrity or resource-isolation violations are hard gate failures.
- Systematic overvalidation/overprovisioning that does not corrupt the current proof may allow the current evidence to finish fail-closed, but must create a conformance failure/incident that blocks declaring the runtime implementation fully accepted until repaired.
- External queue delay, provider outage, or transient hosted-runner scarcity is telemetry/incident evidence, not product failure and not itself a reason to weaken the gate.

11. Verification Throughput as a Quality Attribute

Sounding Line SHALL measure verification cost without turning a performance target into a reason to skip proof. Metrics distinguish queue, setup, execution, teardown, and finalization so the system can optimize the correct layer.

Metric	Required interpretation
Queue time	Time awaiting runner/resource capacity. External queue must not be charged to product implementation.
Runner/bootstrap setup	Checkout, dependency restore/install, generated clients, baseline restore, browser restore.
Suite execution	Time inside actual protected assertions.
Teardown/cleanup	Server/process termination, mutation checks, resource release.
Artifact/evidence finalization	Receipt upload/validation and finalizer execution.
Setup-to-execution ratio	Detects workers whose environment preparation dominates useful proof.

Metric	Required interpretation
Preservation ratio	Shows how much unchanged proof was safely reused after narrow change/main advance.
Over-invalidation ratio	Evidence rerun with no changed dependency. Target trends to zero.
Full-gate count per phase	Repeated full gates without material product change trigger review.

Performance targets are engineering SLOs, not escape hatches

Missing a throughput target must trigger diagnosis and infrastructure improvement. It must never authorize dropping a mandatory contract, skipping an accessibility/security/privacy assertion, or accepting stale evidence.

12. Part I v1.2 Acceptance Additions

- Planner deterministically computes a Minimum Sufficient Evidence Set for every gate.
- Every mandatory obligation resolves to exactly one valid current producer before RELEASE_GO.
- Semantic invalidation dispositions expose machine-readable dependency paths.
- Carry-forward and rebinding are immutable, source-bound, auditable, and finalizer-consumable.
- Test-only, record-only, and unrelated-mainline changes obey explicit invalidation ceilings unless broadened by a dependency explanation.
- Conservative full rerun remains available as a fail-closed fallback but is tagged, justified, and creates implementation debt rather than becoming steady state.
- Resource preparation is part of conformance: undeclared expensive setup is detected.
- Bundled workers may execute multiple suites only if independent per-suite receipt completeness remains exact.
- Gate-specific evidence depth is machine-readable, including a dedicated record-closure gate.
- Runtime conformance evidence is mandatory inside the existing Sounding Line authority chain.
- Performance and preservation metrics are emitted without weakening required proof.
- A reference acceptance scenario proves an optimized run and an unoptimized correct run produce equivalent mandatory decision claims.

Final Part I v1.2 rule

Prove everything that matters. Preserve everything that still means the same thing. Rerun exactly what changed. Treat redundant execution and undeclared setup as verification defects, not as rituals that somehow create confidence.

R. References and Amendment Closure

- Project Sounding Line Part I - Software Verification Architecture, Version 1.0, July 28, 2026.
- Project Sounding Line Part I - Version 1.1 Amendment Edition, August 10, 2026.
- Companion Project Sounding Line Parts II and III, Version 1.0 and Version 1.1 amendments.
- Current repository Sounding Line policy, registry, planner, authority, adapters, finalizer, CI workflows, and accepted project evidence at implementation time.
- August 2026 authoritative-run observations from the concurrent project wave, used as incident evidence rather than as permanent topology assumptions.

All unaffected Version 1.0 and Version 1.1 clauses remain governing. Where Version 1.1 allows manual or broad conservative behavior solely because automation is incomplete, Version 1.2 requires the missing automation to be implemented or the fallback to be explicitly labeled and governed as temporary debt.